Dapper App Local Network Deployment

Troubleshooting & Configuration Documentation

1. Initial Task: Creating a Windows Firewall Inbound Rule

Question:

- Open Windows Defender Firewall with Advanced Security (search in Start menu)
- Click Inbound Rules (left panel) → New Rule (right panel)

where to click?

2. Diagnosis: Network Profile Set to Public

This screenshot confirms the real issue: your server's WiFi network (BEST1997_5GHz 2) is classified as a Public network, and Public networks block all incoming connections by default — which is exactly what's stopping other devices from reaching your app, even before firewall rules for specific apps come into play.

Don't click anything on this page — this Control Panel screen won't let you change the network type. You need the Settings app instead.

Steps to switch the network to Private

1. Press Win + I to open Settings
2. Go to Network & Internet
3. Click on Wi-Fi (or Status, depending on Windows version) → then click on the connected network name (BEST1997_5GHz 2)
4. Under Network profile type, select Private

(On Windows 11, the path is: Settings → Network & Internet → Wi-Fi → click the connected network → Network profile type → Private)

After switching to Private

1. Go back to this same Control Panel page (Windows Defender Firewall) and refresh — it should now show Private networks: Connected instead of Guest/Public
2. Now add the firewall rule for ports 8080/8081 as described earlier (if you haven't already)
3. Test again from your mobile device:

```
http://192.168.1.10:8081
```

This one setting — Public vs Private — is very likely the entire blocker here, since "Public" mode is designed to make your PC invisible to other devices on the same network (useful in cafes/airports, but exactly what's breaking your local deployment access).

3. Adding the Firewall Rule for Ports 8080/8081

Question: "Now add the firewall rule for ports 8080/8081 as described earlier (if you haven't already) — Where to add?"

Here's the exact path to add it:

Steps

1. Press Win key, type wf.msc, hit Enter (this opens Windows Defender Firewall with Advanced Security directly — faster than navigating through Control Panel)
2. In the left panel, click Inbound Rules
3. In the right panel (Actions), click New Rule...
4. A wizard opens — go through it like this:
    - Rule Type: select Port → Next
    - Protocol: select TCP; under "Specific local ports" type:

```
8080,8081
```

    - → Next
    - Action: select Allow the connection → Next
    - Profile: check all three boxes (Domain, Private, Public) → Next
    - Name: type something like IIS Dapper App → Finish

After adding it

1. Confirm the network profile is now set to Private (from the previous step)
2. Test from your mobile device again:

```
http://192.168.1.10:8081
```

If it still doesn't load, run this on the server to double-check IIS is actually listening on that IP/port combo:

```
netstat -ano | findstr :8081
```

Let me know what happens after adding the rule.

4. Connection Refused Error

Observed error (from browser):

```
Hmmm… can't reach this page
192.168.1.10 refused to connect.
Try:
- Search the web for 192 168 1 10
- Checking the connection
- Checking the proxy and the firewall

ERR_CONNECTION_REFUSED
```

Diagnosis and questions asked

A couple of things stand out here — first, which device are you testing from, and did you type the full URL with the port number?

The error message shows just 192.168.1.10 with no port — if you typed that without :8081 at the end, the browser defaults to port 80, where nothing is listening, which would give exactly this "connection refused" error.

Quick checks

1. Make sure you're including the port:

```
http://192.168.1.10:8081
```

Not just 192.168.1.10 alone.

2. Confirm the server's IP hasn't changed

If your server gets its IP via DHCP (not a static IP), it can change after a reboot or reconnect. On the server, open Command Prompt and run:

```
ipconfig
```

Check the IPv4 Address under your active network adapter (WiFi or Ethernet, whichever is actually connected) — confirm it's still 192.168.1.10. If it's different now, that's your whole problem — update the URL, IIS bindings, frontend API URL, and CORS policy to match the new IP.

3. Confirm IIS site is actually still running

Open IIS Manager on the server → check both Dapper_Frontend and Dapper_CRUD sites show Started status, and their app pools are also Started.

4. Test locally on the server first

On the server itself, open a browser and try:

```
http://192.168.1.10:8081
```

If this fails even on the server itself (not from another device), the problem is local — IP mismatch, IIS not running, or binding misconfigured. If it works locally but fails from other devices, then it's back to network/firewall territory.

Information requested

- Which device you're testing from (desktop or mobile)
- Whether you included :8081 in the URL
- What ipconfig shows as the current IP on the server

5. Running ipconfig — First Check

Command entered on the server (user: Sharanya):

```
ipconfig
```

Output:

```
C:\Users\Sharanya>ipconfig
Windows IP Configuration
Wireless LAN adapter Local Area Connection* 1:
   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
Wireless LAN adapter Local Area Connection* 2:
   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
Wireless LAN adapter Wi-Fi:
   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::8a53:6753:bfc:6bfd%17
   IPv4 Address. . . . . . . . . . . : 192.168.1.9
```

```
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : fe80::1%17
                                       192.168.1.1
Ethernet adapter Ethernet:
   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
```

6. Root Cause Found: IP Address Changed

Found it — your server's IP has changed to 192.168.1.9, not 192.168.1.10 anymore. That's exactly why everything's failing — you've been trying to reach an IP the server no longer has.

Fix: Update everything to the new IP

1. Test it first, right now:

```
http://192.168.1.9:8081
```

Try this from the server itself first, then from another device on the same WiFi.

2. Update IIS bindings (both sites)

- IIS Manager → Dapper_Frontend → Edit Bindings → change IP from 192.168.1.10 to 192.168.1.9
- Same for Dapper_CRUD

3. Update frontend code

In your EmployeeService.js:

```
const API_URL = "http://192.168.1.9:8080/api/Employee";
```

Then npm run build → copy fresh dist contents to C:\inetpub\wwwroot\dapper_frontend

4. Update backend CORS

In Program.cs:

```
policy.WithOrigins("http://192.168.1.9:8081")
```

Then republish and recycle the Dapper_CRUD app pool.

Why this keeps happening — fix it permanently

Since this is DHCP-assigned, it can change again anytime (reboot, router restart, lease renewal). Two ways to prevent this going forward:

Option A — Set a static IP on the server (recommended for a "deployed" app)

- Settings → Network & Internet → Wi-Fi → click your network → Edit IP settings → switch from Automatic (DHCP) to Manual, and assign a fixed IP like 192.168.1.9 (or any free address in your range)

Option B — DHCP reservation on your router

- Log into your router's admin panel → find DHCP settings → reserve 192.168.1.9 permanently for this machine's MAC address, so it never changes even with DHCP

Either option means you won't have to re-do all these steps every time the IP shifts. Want me to walk you through setting a static IP?

7. Question: Why Did the IP Address Change?

Your server gets its IP automatically from your router via DHCP (Dynamic Host Configuration Protocol) — the router hands out IP addresses from a pool, and by default these are "leased" temporarily, not permanently assigned to your device.

Common reasons the IP changes

1. DHCP lease expired and renewed — routers assign IPs for a fixed time (the "lease duration," often 24 hours or a few days). When the lease expires and your device reconnects, the router may hand out a different available IP from the pool, especially if the old one got reassigned to another device in the meantime.
2. Device disconnected and reconnected to WiFi — every time your laptop's WiFi disconnects (sleep, restart, moving out of range) and reconnects, it requests a new lease. If the router's pool has shifted, you get a different address.
3. Router itself restarted — if your router rebooted (power cut, ISP reset, manual restart), it often reassigns leases from scratch, which can shuffle IPs across all connected devices.
4. Order of connection matters — DHCP often just hands out the next available IP from its pool. If another device grabbed 192.168.1.10 first, your laptop gets bumped to 192.168.1.9 (or whatever's next free) instead.

Why this matters for what you're doing

Since you're running a server-like deployment (IIS hosting a live app), this is exactly the situation where a stable, unchanging IP matters — servers are typically expected to always be reachable at the same address. Home/office networks using default DHCP settings don't guarantee that, which is why real servers are almost always configured with either:

- A static IP set manually on the machine, or
- A DHCP reservation on the router (still "DHCP," but the router always gives that one device the same IP based on its MAC address)

Since you asked "why," want me to also walk through how to permanently fix this (static IP or router reservation), so you don't have to redo all these steps every time your laptop reconnects to WiFi?

8. Request: Keep 192.168.1.10 Permanent

User: "I want to keep 192.168.1.10 as permanent — how to do it? But IP address never changes?"

Good news: 192.168.1.10 is free right now (nothing responded when you tried it), so you can claim it. Two ways to do this — here's both, pick one.

To answer "does IP address never change?"

By default with DHCP, yes it can change (as explained before). It only stays fixed if you explicitly configure it to — either on the device itself (static IP) or on the router (DHCP reservation). Neither happens automatically; you have to set it up.

Option A: Set a Static IP on your laptop (simplest, done in 2 minutes)

1. Press Win + I → Network & Internet → Wi-Fi
2. Click on your connected network (BEST1997_5GHz 2)
3. Scroll to IP settings → click Edit
4. Change dropdown from Automatic (DHCP) to Manual
5. Turn IPv4 toggle On

6. Fill in:
    - IP address: 192.168.1.10
    - Subnet mask (or "Subnet prefix length"): 255.255.255.0 (or 24 if it asks for prefix length)
    - Gateway: 192.168.1.1 (from your ipconfig output)
    - Preferred DNS: 192.168.1.1 (or 8.8.8.8 for Google DNS — either works)
7. Click Save

Run ipconfig again afterward to confirm it now shows 192.168.1.10.

Downside of this method: if your router happens to hand out 192.168.1.10 to some other device via DHCP while your laptop is off, you'll get an IP conflict when you reconnect (both devices claiming the same address, causing connectivity issues for one or both). This is why static-on-device is fine for quick setups, but not bulletproof.

Option B: DHCP Reservation on your router (more robust, recommended)

This keeps DHCP "automatic" but tells the router "always give this specific device (identified by its MAC address) the IP 192.168.1.10," every time. No conflict risk, since the router manages it centrally.

1. Find your laptop's MAC address: run ipconfig /all and look for Physical Address under the WiFi adapter
2. Log into your router's admin page (usually 192.168.1.1 in a browser, credentials often printed on the router itself)
3. Find DHCP Settings / Address Reservation / Static Leases (naming varies by router brand)
4. Add a new reservation: enter your laptop's MAC address → assign IP 192.168.1.10
5. Save, then restart your laptop's WiFi connection (disable/enable) or reboot the router for it to apply

This is the more "correct" way for anything meant to behave like a stable server.

Once either is done and confirmed via ipconfig, you won't need to update your IIS bindings, frontend URL, or CORS policy again — 192.168.1.10 will be your permanent answer.

9. Static IP Attempt Failed — WiFi Disconnected

User followed Option A step by step (as documented above) and reported:

"Wifi is not getting connected after doing this so i will roll back and turn on DHCP Automatic"

That's fine — rolling back is the right call, especially since a manual static IP on a shared/public WiFi (hotel, office, campus-type network) often fails because the router may reserve that IP for something else, enforce MAC filtering, or use a different subnet than you assumed.

To roll back

1. Win + I → Network & Internet → Wi-Fi
2. Click your network (BEST1997_5GHz 2)
3. Scroll to IP settings → Edit
4. Change back from Manual to Automatic (DHCP)
5. Click Save

Your WiFi should reconnect normally within a few seconds. Run ipconfig again to confirm you're back on a DHCP-assigned address (might be 192.168.1.9 again, or something else).

Better path forward: DHCP Reservation on the router (Option B)

Since manual static IP isn't cooperating on this network, Option B (router-side DHCP reservation) is actually the safer and more reliable choice here — it keeps your laptop on automatic DHCP (so it always connects fine), but the router itself always hands out the same IP to your laptop specifically.

Once you're back online, if you want, I can walk you through finding your router's admin panel and setting up that reservation — but heads up: if this WiFi belongs to a shared/managed network (hotel, PG, office, campus), you likely won't have admin access to the router, in which case DHCP reservation isn't an option to you at all, and you'd just have to accept the IP may change and update it when needed.

Is this your own home router, or a shared/managed network?

10. Rollback Completed

User: "Done i hav roll backed all the changes now tell what to do?"

Good. Let's get back to a working state.

1. Confirm your current IP

Open Command Prompt and run:

```
ipconfig
```

Check the IPv4 Address under Wireless LAN adapter Wi-Fi — note whatever it shows now (could be 192.168.1.9 again, or something else).

2. Update everything to match that current IP

Since we don't know yet if it's .9, .10, or something else, once you confirm it, update these three places:

a. IIS bindings (both sites)

- IIS Manager → Dapper_Frontend → Edit Bindings → set IP to the current one
- Same for Dapper_CRUD

b. Frontend EmployeeService.js

```
const API_URL = "http://<current-ip>:8080/api/Employee";
```

Then npm run build → copy fresh dist into C:\inetpub\wwwroot\dapper_frontend

c. Backend Program.cs CORS

```
policy.WithOrigins("http://<current-ip>:8081")
```

Then republish + recycle Dapper_CRUD app pool.

3. About making it permanent

Since this is a shared/public-type network (given the "BEST1997" WiFi name and the manual-IP failure), you likely won't have router admin access for a DHCP reservation either. In that case, realistically:

- The IP may keep changing occasionally on this kind of network, and there isn't a way to fully lock it down from your side
- The practical approach becomes: whenever mobile/other-device access stops working, just re-run ipconfig, and repeat the 3 updates above

Not ideal, but it's a common reality on networks you don't control. If this project moves beyond local testing, hosting on a proper server with a static IP (cloud VM, or your own router at home) would be the real fix.

Run ipconfig now and tell me the current IP — I'll help you get everything pointed at it correctly.

11. Second ipconfig Check

Command entered on the server (user: Sharanya):

```
ipconfig
```

Output:

```
C:\Users\Sharanya>ipconfig
Windows IP Configuration
Wireless LAN adapter Local Area Connection* 1:
   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
Wireless LAN adapter Local Area Connection* 2:
   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
Wireless LAN adapter Wi-Fi:
   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::8a53:6753:bfc:6bfd%17
   IPv4 Address. . . . . . . . . . . : 192.168.1.9
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : fe80::1%17
                                       192.168.1.1
Ethernet adapter Ethernet:
   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
```

Question asked: "now what to do"

12. Final Configuration Update to 192.168.1.9

Good — it's back to 192.168.1.9. Let's update everything to this IP.

1. IIS Bindings (both sites)

Frontend (Dapper_Frontend):

- IIS Manager → click Dapper_Frontend → Bindings (right panel, under "Edit Site") → select the binding → Edit
- Change IP address to 192.168.1.9 → OK

Backend (Dapper_CRUD):

- Same steps → click Dapper_CRUD → Bindings → Edit → change IP to 192.168.1.9 → OK

2. Update frontend code

In EmployeeService.js:

```
const API_URL = "http://192.168.1.9:8080/api/Employee";
```

Then:

```
npm run build
```

Copy the fresh dist folder contents into C:\inetpub\wwwroot\dapper_frontend (overwrite old files).

3. Update backend CORS

In Program.cs:

```
policy.WithOrigins("http://192.168.1.9:8081")
```

Then:

- Republish: dotnet publish -c Release -o ./publish (or VS Publish)
- Copy output to your backend's IIS folder (overwrite old files)
- IIS Manager → Application Pools → right-click Dapper_CRUD → Recycle

4. Test

From the server itself first:

```
http://192.168.1.9:8081
```

Then from your mobile (same WiFi):

```
http://192.168.1.9:8081
```

Go through these one at a time and let me know once done, or if you hit an error at any step.

13. Question: Can We Change 192.168.1.9 Back to the Old One?

Not reliably, no — that's essentially what you tried before with the static IP (Option A), and it broke your WiFi connection. Here's why, and what your real options are.

Why it didn't work last time

When you tried manually setting 192.168.1.10, WiFi failed to connect. This usually happens because:

- The router's DHCP pool might already have .10 reserved/leased to another device (even if it didn't respond to a browser test, it could still be "reserved" or blocked by the router)
- On many shared/managed networks (hotel, PG, office-style WiFi — which "BEST1997" sounds like), routers often block manually-set static IPs entirely as a security measure, only allowing DHCP-assigned addresses
- MAC filtering or client isolation could be enforced, which prevents manual IP assignment specifically while still allowing DHCP

This isn't a mistake you made — it's a router-side restriction that's very common on non-home networks.

Your realistic options

Option 1 — Just use 192.168.1.9 (current, working IP)

Simplest path forward. Update the 3 places (IIS bindings, frontend URL, backend CORS) to .9 as I outlined, and it'll work right now without fighting the network.

Option 2 — Try static .10 again, but properly

Only worth trying if you want to debug further:

- Try a different free IP first (not .10 specifically) — e.g. .50 or .100 — to test if manual IP assignment works on this network at all, regardless of address

- If even that fails to connect, the network itself is blocking manual/static IPs, and no specific IP (including .10) will work manually

Option 3 — Router reservation for .10

Only possible if you have admin access to this router. Given it's likely a shared network, this is probably not available to you.

Suggestion

Given the WiFi failure you already hit, go with Option 1 — just use .9, the IP that's actually working right now. Trying to force .10 again risks losing WiFi connectivity again for uncertain benefit. Want to proceed with updating everything to .9?

14. Question: Should the Firewall Rule and Private Network Setting Be Rolled Back?

User: "Should i delete this and also rollback private to public network under wifi?"

No, keep both as they are — don't delete or roll those back.

Firewall rule (IIS Dapper App)

Leave this in place. It's not tied to any specific IP — it just allows traffic on ports 8080/8081 regardless of what IP address gets used. You'll need this rule either way, whether you're on .9 or .10.

Private network setting

Leave this as Private too. This was a separate, necessary fix — Public network mode blocks incoming connections by default, which was part of why your mobile couldn't reach the server. Switching back to Public would break connectivity again for other devices, regardless of which IP you use.

What actually needs rolling back (already done)

Only the static IP configuration was the problematic change — and you already reverted that back to Automatic (DHCP), which is correct.

So current correct state should be:

- ✅Network profile: Private
- ✅Firewall rule: IIS Dapper App (allowing 8080/8081) — keep it
- ✅IP assignment: Automatic (DHCP) — already rolled back
- □ Still pending: update IIS bindings, frontend URL, and backend CORS to the current IP (192.168.1.9)

Let's proceed with updating those three things to .9 now — want to go ahead with that?

15. Successful Test Confirmation

Test performed:

From the server itself first:

```
http://192.168.1.9:8081
```

Then from your mobile (same WiFi):

```
http://192.168.1.9:8081
```

User result: "I can see it from both mobile and from my laptop"

Confirmation of working state

That confirms everything is working correctly now:

- ✓Frontend and backend bound to 192.168.1.9
- ✓Firewall rule allowing the ports
- ✓Network profile set to Private
- ✓Reachable from both the server itself and your mobile device over WiFi

One thing worth double-checking

Since the table needs backend data too, open your mobile browser's or laptop's dev tools (if on laptop) and confirm the Employee table is actually populating with data (not just the page shell loading) — this verifies the backend API call and CORS are also working correctly from the new IP, not just that the frontend static files are reachable.

If the table shows data on both devices, you're fully done with this deployment. □

If it's just showing the empty table (headers only, no rows), that would mean the frontend loaded fine but the API call is still failing — let me know if that's the case.

Summary of Final State

- Network profile: Private
- Firewall rule "IIS Dapper App": allows TCP ports 8080 and 8081 across Domain, Private, and Public profiles
- IP assignment: Automatic (DHCP)
- Current working server IP: 192.168.1.9
- IIS site bindings (Dapper_Frontend, Dapper_CRUD): updated to 192.168.1.9
- Frontend EmployeeService.js API_URL: updated to http://192.168.1.9:8080/api/Employee
- Backend Program.cs CORS policy: updated to http://192.168.1.9:8081
- Verified reachable from server itself and from mobile device on the same WiFi